

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune a pretrained CNN model (MobileNetV2) on CIFAR-10.
- Compare classification performance before and after fine tuning.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning using MobileNetV2.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features directly from raw image data. The evolution of CNN architectures has been driven by two goals: improving accuracy and reducing computational cost. This experiment studies the evolution from LeNet-5 to MobileNetV2, with a practical transfer learning implementation using MobileNetV2 on the CIFAR-10 dataset.

4. Background Theory – Evolution of CNN Architectures

4.1 LeNet-5

LeNet-5 (LeCun, 1998) was the first practical CNN, designed for handwritten digit recognition. It has 7 layers and approximately 60K parameters, using average pooling and sigmoid activation. It proved that networks could learn image features without handcrafted feature extraction, but is too shallow for complex natural images.

4.2 AlexNet

AlexNet won the 2012 ImageNet Challenge, reducing top-5 error from 26% to 15%. Its key innovations were ReLU activation (avoiding vanishing gradients from Sigmoid/Tanh), Dropout regularization, GPU training, and data augmentation. It has 8 layers and 61M parameters.

4.3 VGG16

VGG16 demonstrated that stacking many small 3×3 filters outperforms fewer large filters. Two stacked 3×3 layers match a 5×5 receptive field with fewer parameters and an extra non-linearity. It has 16 layers and 138M parameters, mostly in its large fully connected layers.

4.4 GoogleNet (Inception Network)

GoogleNet introduced the Inception Module, applying 1×1 , 3×3 , 5×5 convolutions and pooling in parallel, then concatenating results. This captures multi-scale features. 1×1 convolutions reduce channel count before larger filters, cutting parameters dramatically. Result: 22 layers, only 6.8M parameters.

4.5 ResNet

ResNet solved the vanishing gradient problem using skip connections:

$$\text{Output} = F(x) + x$$

where $F(x)$ is the residual mapping and x is the identity. Gradients flow directly backward through the skip connection, enabling networks of 50, 101, or more layers. ResNet50 has 25.6M parameters.

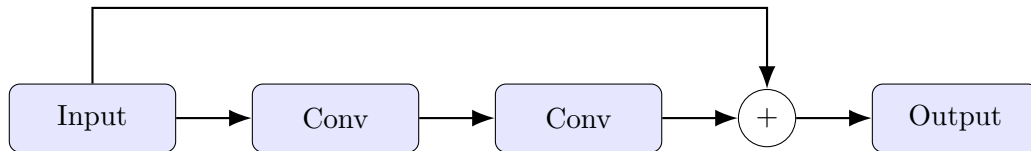


Figure 1: Residual Block in ResNet

4.6 MobileNetV2

MobileNetV2 (Sandler et al., 2018) was designed for mobile and embedded applications. Its core building block is the **Inverted Residual with Linear Bottleneck**, which:

1. Expands the number of channels with a 1×1 convolution (expansion layer)
2. Applies a **depthwise separable convolution** — one filter per input channel
3. Projects back to fewer channels with another 1×1 convolution

Standard convolution applies every filter to every input channel simultaneously. Depthwise separable convolution splits this into two steps: depthwise convolution (one filter per channel) followed by point-wise convolution (1×1). This reduces computation by a factor of approximately 8–9× compared to standard convolution.

MobileNetV2 has approximately 3.4M parameters — far fewer than VGG16 (138M) or ResNet50 (25.6M) — while achieving competitive accuracy on ImageNet. This efficiency makes it ideal for transfer learning when compute is limited.

4.7 Dilated Convolution

Dilated convolution increases the receptive field by inserting gaps between kernel weights (dilation rate $D > 1$), without increasing parameters or losing spatial resolution. Applications include semantic segmentation, medical imaging, and satellite imagery.

4.8 Transpose Convolution

Transpose convolution performs learnable upsampling, increasing spatial dimensions. Used in autoencoders, GANs, and semantic segmentation decoders.

5. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning
MobileNetV2	53	3.4M	Depthwise Separable Convolution

6. Transfer Learning

Transfer learning reuses a CNN pretrained on ImageNet for a new, smaller dataset. The standard workflow is:

1. Load a pretrained model with ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add new Dense layers for the target task.
5. Train the new head (Phase 1 – frozen backbone).
6. Unfreeze the last convolutional block and fine tune at a low learning rate (Phase 2).

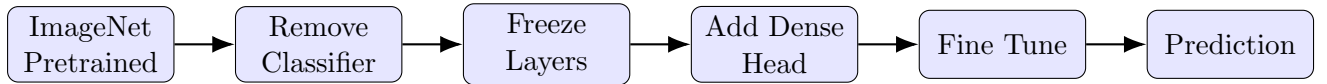


Figure 2: Transfer Learning Workflow

7. Dataset

Property	Value
Dataset	CIFAR-10
Training Images	50,000
Testing Images	10,000
Native Image Size	$32 \times 32 \times 3$
Resized Image Size	$96 \times 96 \times 3$
Classes	10

Images were resized from 32×32 to 96×96 to balance transfer learning effectiveness with available GPU memory constraints. MobileNetV2 officially supports input sizes as small as 32×32 ; using 96×96 preserves meaningful spatial structure across all pooling layers while reducing memory consumption by approximately $5.5\times$ compared to the standard 224×224 input. Pixel values were normalised using `preprocess_input` which scales to the range $[-1, 1]$ as required by MobileNetV2.

8. Model Architecture

Component	Details
Backbone	MobileNetV2 (ImageNet pretrained)
Frozen Layers	All except last 20 layers during fine tuning
Global Avg Pooling	Reduces feature maps to 1280-dim vector
Dense Layer	256 units, ReLU activation
Dropout	Rate = 0.3
Output Layer	10 units, Softmax
Loss Function	Categorical Cross-Entropy
Phase 1 Optimizer	Adam, lr = 0.001
Phase 2 Optimizer	Adam, lr = 0.0001
Phase 1 Epochs	10
Phase 2 Epochs	5
Batch Size	32

Parameter Counts

Stage	Trainable Parameters
Total parameters (entire model)	2,588,490
Phase 1 – frozen backbone (head only)	330,506
Phase 2 – after unfreezing last 20 layers	~2,257,984

9. Experimental Procedure

Task 1: Dataset Preparation

CIFAR-10 loaded using `tf.keras.datasets.cifar10`. Images resized to 96×96 and preprocessed with `preprocess_input`. Ten sample images displayed. Class distribution plotted.

Task 2: Transfer Learning Setup

MobileNetV2 loaded with ImageNet weights (`include_top=False`). All base layers frozen. New head added: GlobalAveragePooling2D $\rightarrow$ Dense(256, ReLU) $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(10, Softmax).

Task 3: Phase 1 Training (Frozen Backbone)

Compiled with Adam (lr=0.001), categorical cross-entropy loss. Trained for 10 epochs with batch size 32.

Task 4: Fine Tuning

Last 20 layers of MobileNetV2 unfrozen. Recompiled with Adam (lr=0.0001). Trained for 5 further epochs.

Task 5: Model Evaluation

Evaluated using accuracy, precision, recall, F1-score, confusion matrix, and classification report on the full 10,000-image test set.

10. Hyperparameter Study

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial (last 20 unfrozen)

11. Mandatory Plots

11.1 Sample CIFAR-10 Images

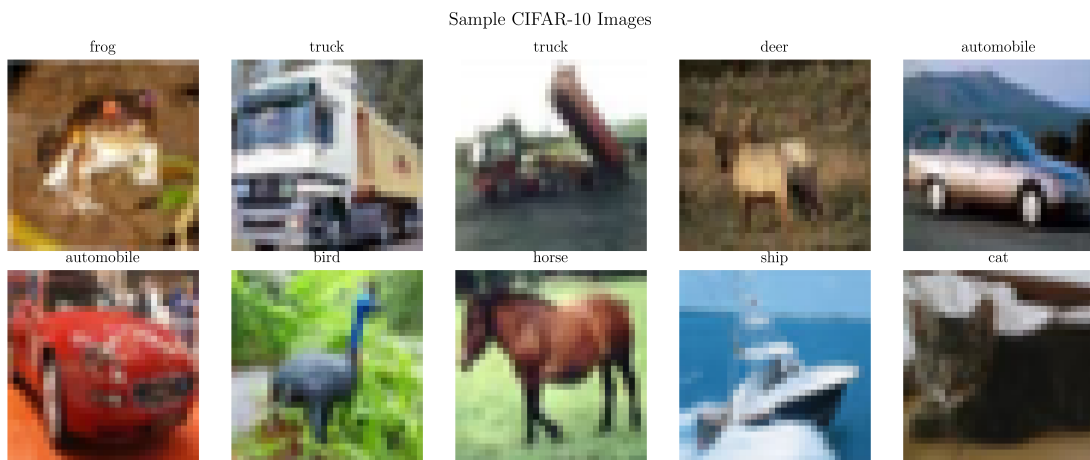


Figure 3: Ten sample images from the CIFAR-10 training set with class labels.

Inference: CIFAR-10 has 10 visually distinct classes with 32x32 RGB images and also they have perfect class distribution.

11.2 Class Distribution

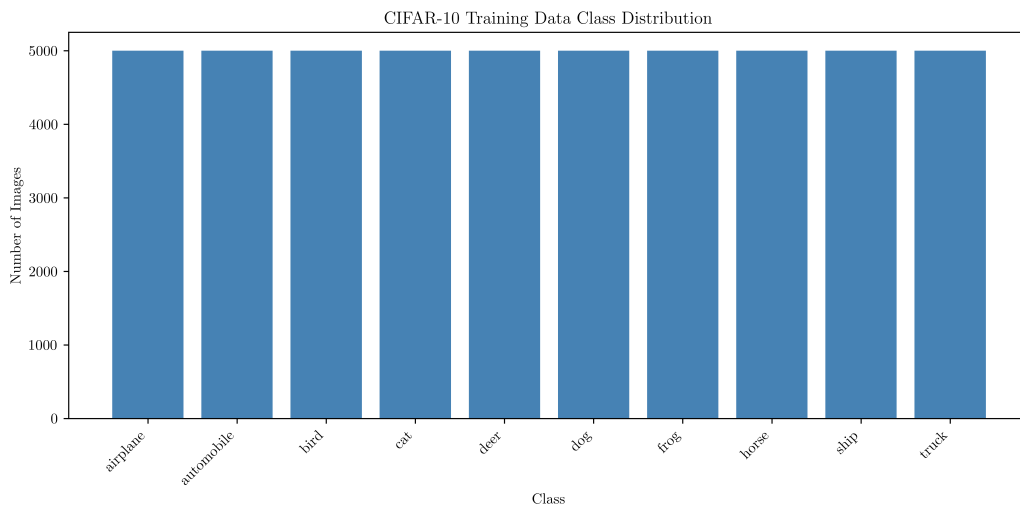


Figure 4: Class distribution of the CIFAR-10 training set.

Inference: CIFAR-10 has a perfectly balanced distribution with 5000 training images per class so there is no question of model imbalance because of dataset imbalance.

11.3 Training Accuracy (Frozen Backbone)

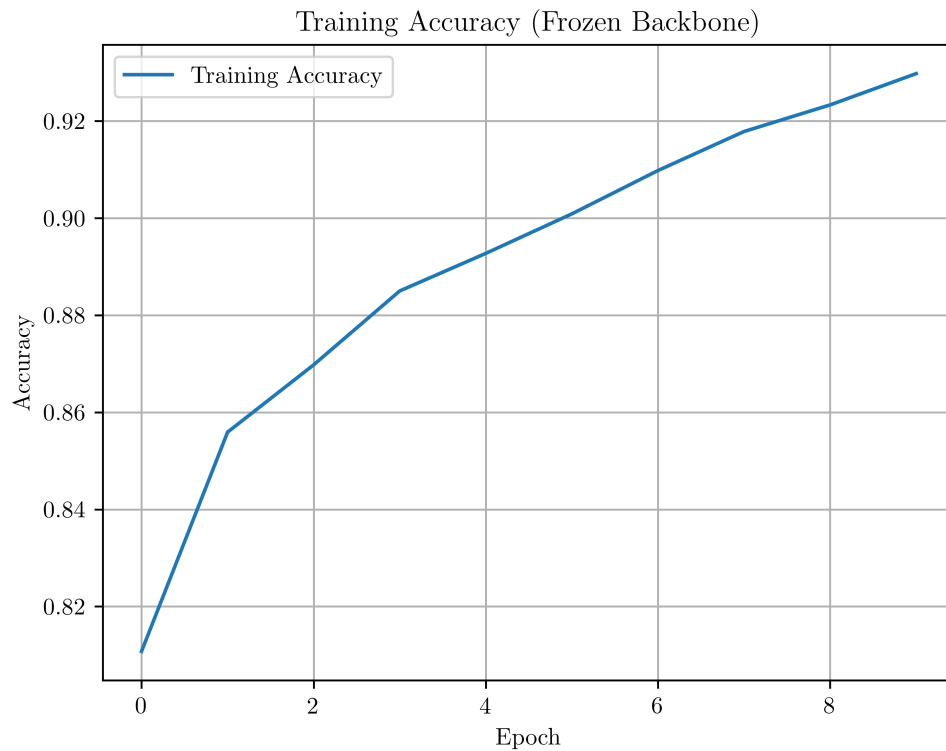


Figure 5: Training accuracy over 10 epochs, frozen backbone phase.

Inference: Training accuracy increases steadily from around 0.81 at epoch 1 to 0.93 by epoch 10 which shows the frozen MobileNetV2 backbone is already extracting useful features and the head is learning effectively.

11.4 Validation Accuracy (Frozen Backbone)

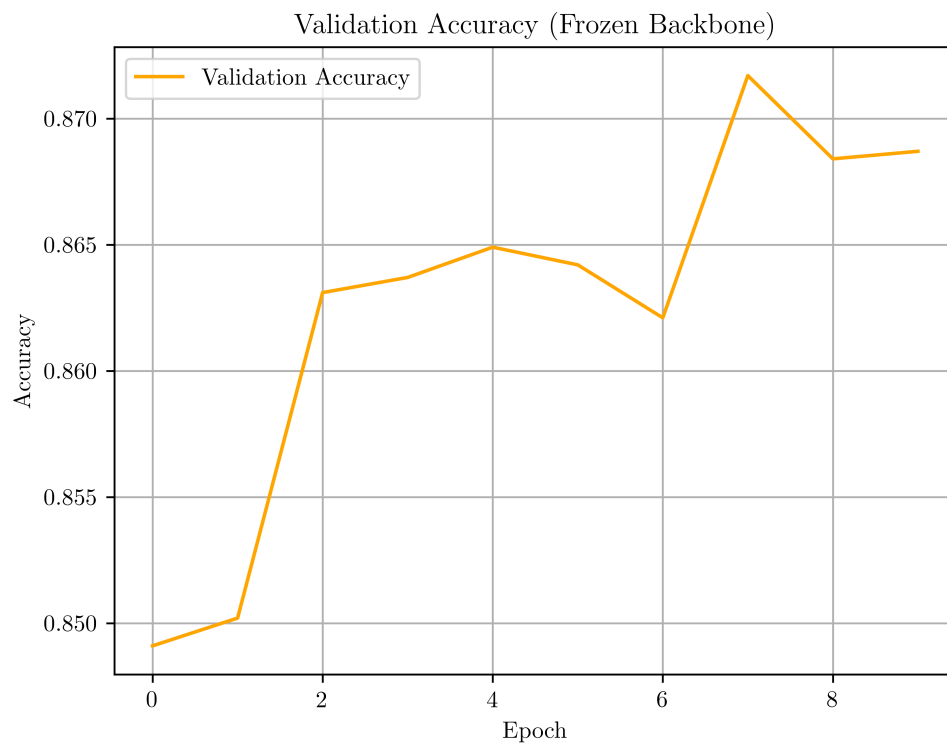


Figure 6: Validation accuracy over 10 epochs, frozen backbone phase.

Inference: Validation accuracy rises from 0.85 at epoch 1 and plateaus around 0.87 from epoch 5 onwards which shows that the backbone saturates within that region.

11.5 Training Loss (Frozen Backbone)

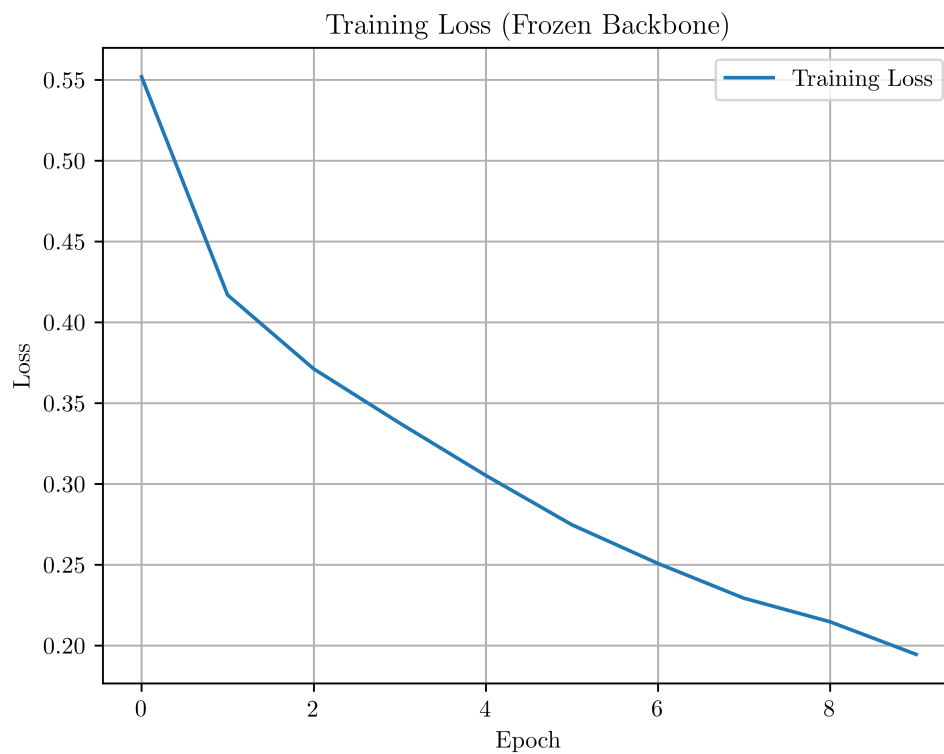


Figure 7: Training loss over 10 epochs, frozen backbone phase.

Inference: Training loss drops sharply from 0.55 in epoch 1 to 0.19 by epoch 10 which essentially says that the Adam optimizer is effectively reducing the objective loss function.

11.6 Validation Loss (Frozen Backbone)



Figure 8: Validation loss over 10 epochs, frozen backbone phase.

Inference: Validation loss drops to around 0.42 by epoch 3 and then starts oscillating which means that convergence may occur slower so dropout and other regularization techniques should be used.

11.7 Confusion Matrix (Before Fine Tuning)

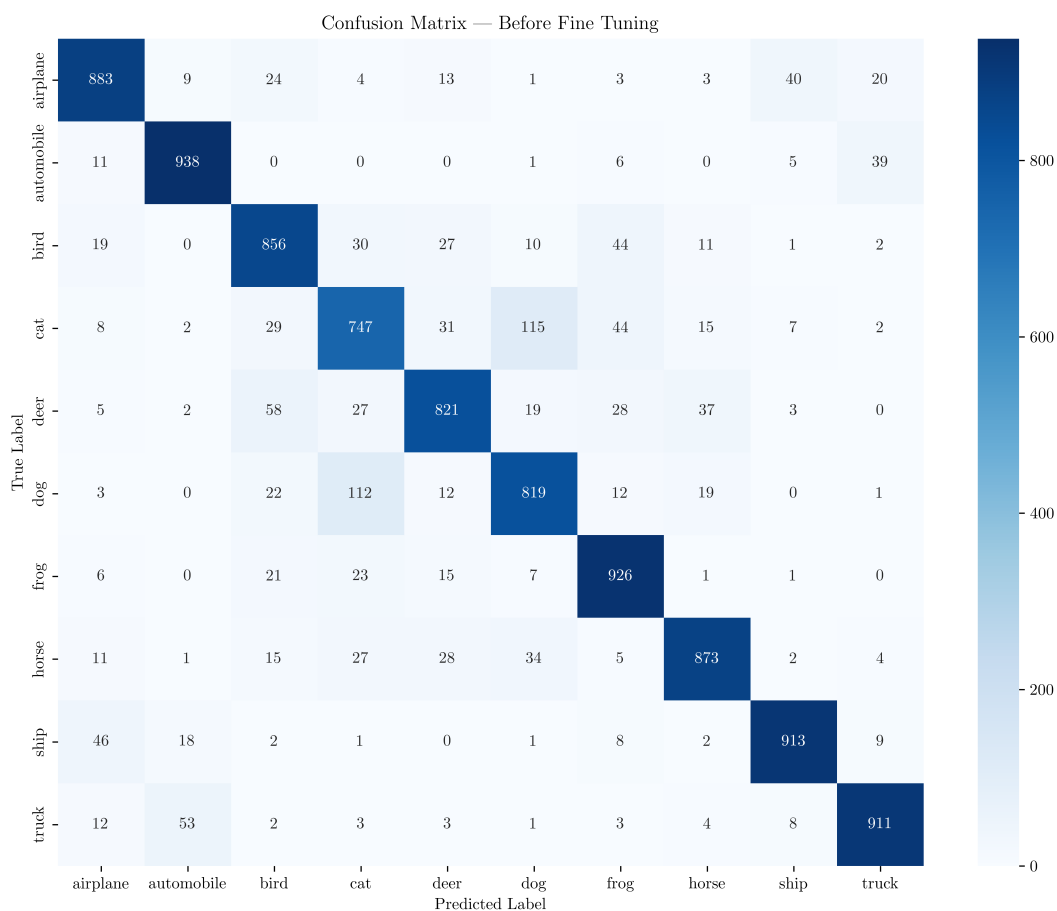


Figure 9: Confusion matrix before fine tuning – Phase 1 frozen backbone predictions on test set.

Inference: The confusion matrix before fine tuning shows cat and dog have the most misclassifications between each other which makes sense given their visual similarity at 32x32 resolution. But, looking at the other classes, they are pretty good in terms of classification quality.

11.8 Fine Tuning Accuracy

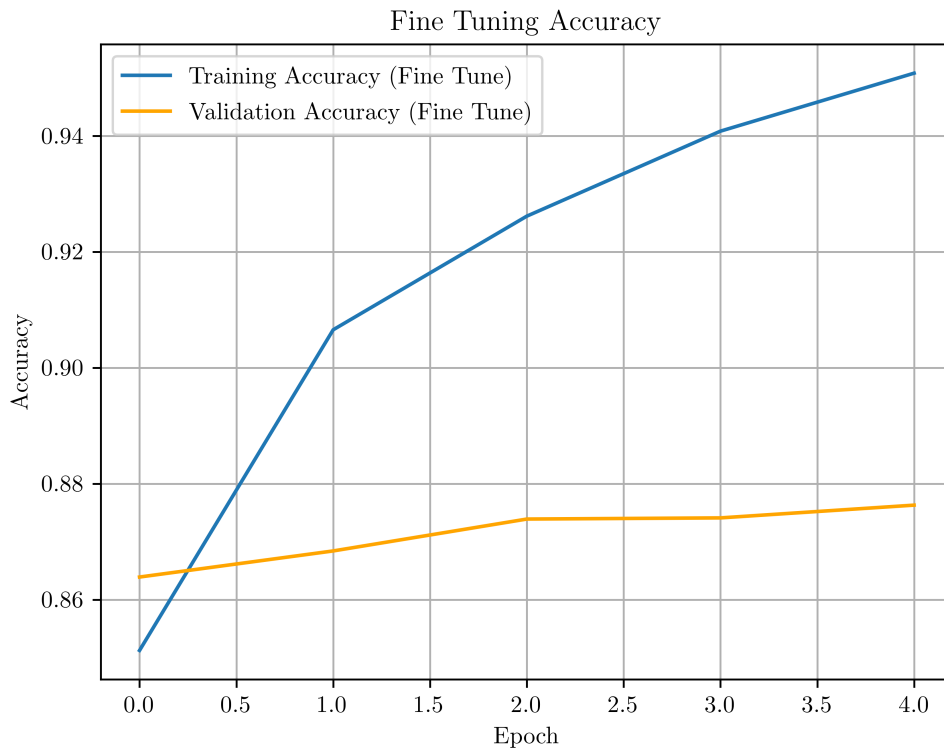


Figure 10: Training and validation accuracy during fine tuning (5 epochs, lr=0.0001).

Inference: Fine tuning accuracy shows a reset at epoch 1 as the unfrozen layers start learning from scratch at a lower learning rate but then rises to 0.95 which means that accuracy has reached a good point of convergence.

11.9 Before vs After Fine Tuning

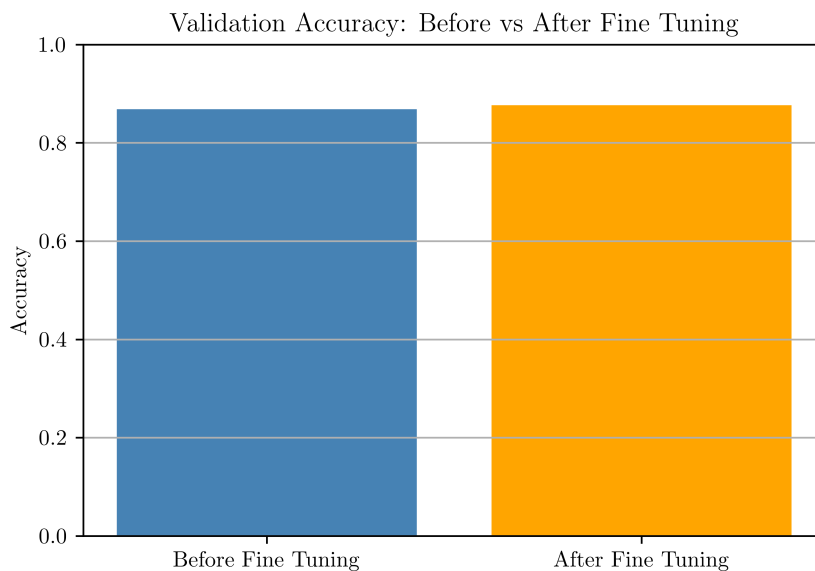


Figure 11: Validation accuracy before and after fine tuning.

Inference: After fine tuning validation accuracy improved by 0.76 percentage points from 86.87 to 87.63 which confirms that making sure that the last 20 layers being unfrozen makes it better in terms

of fine tuning the model to boost accuracy.

11.10 Confusion Matrix (After Fine Tuning)

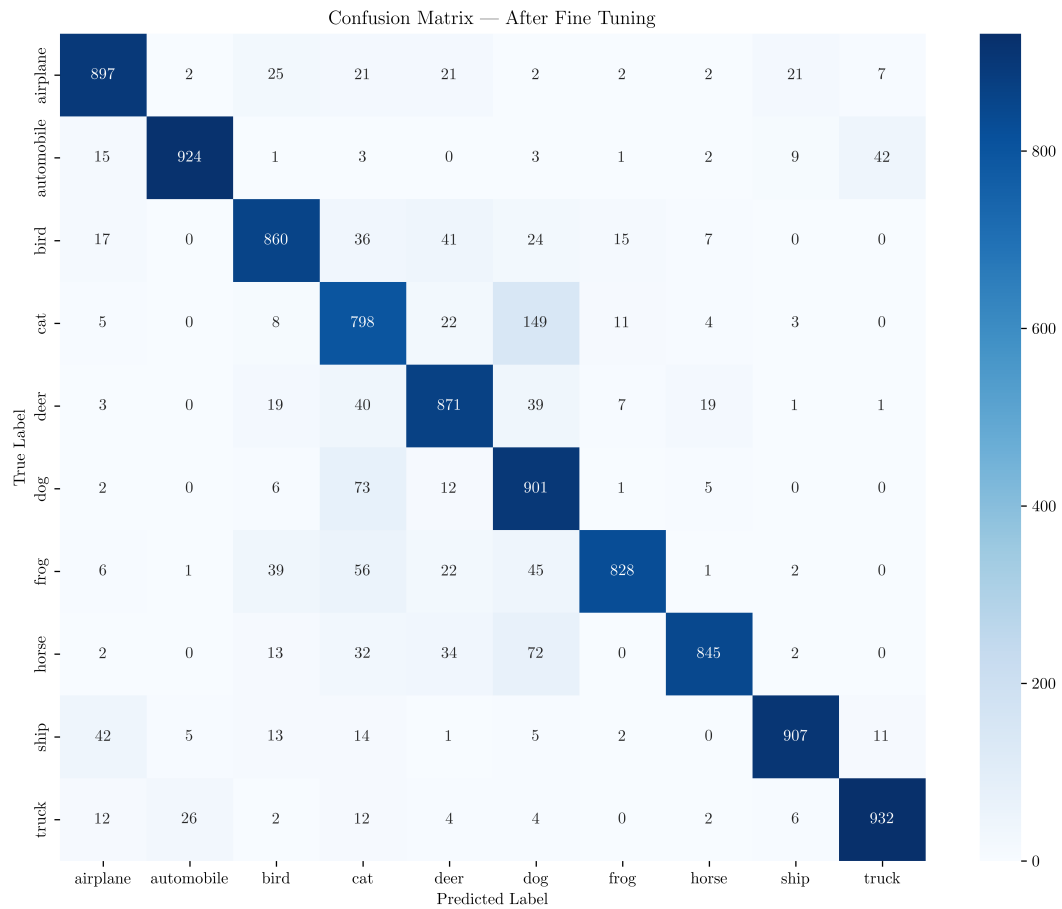


Figure 12: Confusion matrix for the fine-tuned MobileNetV2 on the CIFAR-10 test set.

Inference: The confusion matrix after fine tuning shows improved classification across most classes with cat and dog still showing the most confusion which is expected given their visual similarity at this resolution.

11.11 Misclassified Images



Figure 13: Eight misclassified test images with true and predicted labels.

Inference: The misclassified images are mostly visually ambiguous samples. Like for example, if a person were to be given photos that are blurred between a cat and dog they may or may not be able to properly say so the model struggles with examples like this.

12. Results

12.1 Phase Comparison

Stage	Train Acc	Val Acc	Train Loss	Val Loss
Frozen Backbone (epoch 10)	92.98%	86.87%	0.1946	0.4693
Fine Tuned (epoch 5)	95.08%	87.63%	0.1402	0.5097

12.2 Test Set Performance Metrics

Metric	Value
Test Accuracy	87.63%
Precision (macro)	0.88
Recall (macro)	0.88
F1-Score (macro)	0.88
Total Training Time	388.0 s (Phase 1: 229.0 s, Phase 2: 159.0 s)
Total Parameters	2,588,490

12.3 Classification Report (Before Fine Tuning)

Class	Precision	Recall	F1-score	Support
Airplane	0.88	0.88	0.88	1000
Automobile	0.92	0.94	0.93	1000
Bird	0.83	0.86	0.84	1000
Cat	0.77	0.75	0.76	1000
Deer	0.86	0.82	0.84	1000
Dog	0.81	0.82	0.82	1000
Frog	0.86	0.93	0.89	1000
Horse	0.90	0.87	0.89	1000
Ship	0.93	0.91	0.92	1000
Truck	0.92	0.91	0.92	1000
Macro avg	0.87	0.87	0.87	10000

12.4 Classification Report (After Fine Tuning)

Class	Precision	Recall	F1-score	Support
Airplane	0.90	0.90	0.90	1000
Automobile	0.96	0.92	0.94	1000
Bird	0.87	0.86	0.87	1000
Cat	0.74	0.80	0.77	1000
Deer	0.85	0.87	0.86	1000
Dog	0.72	0.90	0.80	1000
Frog	0.96	0.83	0.89	1000
Horse	0.95	0.84	0.90	1000
Ship	0.95	0.91	0.93	1000
Truck	0.94	0.93	0.94	1000
Macro avg	0.88	0.88	0.88	10000

12.5 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Source
LeNet-5	60K	–	Literature
AlexNet	61M	–	Literature
VGG16	138M	–	Literature
GoogleNet	6.8M	–	Literature
ResNet50	25.6M	–	Literature
MobileNetV2	2.59M	87.63	This experiment

9. Discussion

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet was the first CNN to win ImageNet comp by a huge margin in 2012 and it proved that deep networks with GPUs and ReLU activations can beat the norm of hand crafted features which

were during those days, the tradition. It also introduced dropout for regularization which became standard practice.

2. Why does VGG16 use only 3×3 convolution filters?

Two 3×3 filters stacked give the same receptive field as one 5×5 filter but with fewer parameters and more non-linearity since each has its own ReLU. This makes the network deeper with less computational cost while still capturing the same spatial patterns.

3. Explain the advantages of the Inception module.

The Inception module runs multiple filter sizes in parallel on the same input and concatenates the results so the network can capture features at different scales without having to choose one filter size. It also uses 1×1 convolutions to reduce depth before the expensive 3×3 and 5×5 convolutions which cuts down parameters significantly.

4. What is the purpose of residual learning?

In very deep networks gradients vanish before reaching the early layers making training difficult. Residual connections skip layers and add the input directly to the output so the gradients can flow back without the vanishing effect, allowing networks with hundreds of layers to train properly.

5. Differentiate LeNet and ResNet.

LeNet is a shallow 5-layer network designed for simple digit recognition with only 60K parameters and no residual connections. ResNet uses certain connections like skip connections to train networks with hundreds of layers and millions of parameters solving the vanishing gradient problem

6. What is Transfer Learning?

Transfer learning is when you take a model that was already trained on a large dataset like ImageNet and reuse its weights as a starting point for a new task. We reduce computation crunch as we are reusing weights for a certain task and it helps to bring optimal performance.

7. Why is fine tuning required?

Fine tuning is needed because the pretrained model learned features for ImageNet which may not perfectly match the new dataset. By unfreezing the later layers and retraining at a very small learning rate, the backbone can adapt its higher-level features to the new domain which is why accuracy improved from 86.87 to 87.63 percent in this experiment.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution spreads the filter out by inserting gaps between its elements so it covers a larger area without increasing the number of parameters, which is useful for capturing context. Transpose convolution goes the other way and upsamples a smaller feature map to a larger one which is used in tasks like segmentation where you need to restore the original image resolution.

9. Why do pretrained models converge faster?

A pretrained backbone essentially knows the basics of the dataset like where the edges are, where the contours lie so they converge faster in that regard but they don't guarantee optimal performance.

10. Compare the computational complexity of LeNet and ResNet.

LeNet has about 60K parameters and runs in a fraction of a second on CPU since it only has 5 layers with small filters designed for 32×32 grayscale images. ResNet-50 has 25.6M parameters and requires GPU acceleration due to its depth of 50 layers with residual blocks. So the trade off comes to LeNet being used for basic tasks with grayscales but ResNet was the evolutionary big brother of a model that can handle more complex

14. Conclusion

MobileNetV2 transfer learning was applied to CIFAR-10 with images resized to 96x96, achieving 86.87 percent validation accuracy after Phase 1 with the frozen backbone and 87.63 percent after fine tuning the last 20 layers at a lower learning rate — an improvement of 0.76 percentage points. The final test accuracy was 87.63 percent with a macro-averaged precision, recall and F1-score of 0.88. Only 330,506 out of 2,588,490 parameters were trained in Phase 1 which shows that pretrained features from ImageNet transfer well to CIFAR-10 even at a reduced input resolution. Cat and dog remained the hardest classes to separate due to their visual similarity at 32x32 native resolution. The conclusion is that freezing the backbone and implementing transfer learning actually helps to make the model better and that MNV2 is the better model for this type of task.

15. References

1. Yann LeCun et al., *Gradient-Based Learning Applied to Document Recognition*, IEEE, 1998.
2. Alex Krizhevsky et al., *ImageNet Classification with Deep CNNs*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He et al., *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Mark Sandler et al., *MobileNetV2: Inverted Residuals and Linear Bottlenecks*, CVPR, 2018.
7. Ian Goodfellow et al., *Deep Learning*. MIT Press, 2016.
8. TensorFlow Documentation: <https://www.tensorflow.org>
9. Keras Documentation: <https://keras.io>